
MYSQL FUNDAMENTALS: DAY 4

AGGREGATION, GROUPING, HAVING, AND CONDITIONAL LOGIC

1. RECAP

Until now, students have been working at the row level:

- Fetching individual records
- Filtering rows

Today we move to **analytical thinking**:

- Summarizing data
- Grouping data
- Applying conditions on groups
- Creating derived logic using **CASE**

This is the foundation of data analysis in SQL.

2. AGGREGATE FUNCTIONS

Aggregate functions operate on multiple rows and return a single value.

2.1 COUNT()

```
SELECT COUNT(*) FROM employees;
```

- **COUNT(*)** → Counts all rows, including NULL values.

```
SELECT COUNT(salary) FROM employees;
```

- Counts only **NON-NULL** salary values.

2.2 SUM()

```
SELECT SUM(salary) FROM employees;
```

- Adds all salary values and ignores NULL values.

2.3 AVG()

```
SELECT AVG(salary) FROM employees;
```

- Calculates average salary and ignores NULL values.

2.4 MIN() & 2.5 MAX()

`SELECT MIN(salary) FROM employees;` → Returns smallest salary. `SELECT MAX(salary) FROM employees;` → Returns highest salary.

3. GROUP BY (VERY IMPORTANT)

Used to group rows that have the same values in specified columns.

3.1 BASIC GROUP BY

`SELECT department, AVG(salary) FROM employees GROUP BY department;`

Step-by-step Execution:

1. Rows are divided into groups based on **department**.
2. **AVG(salary)** is calculated for each specific group.
3. One row per group is returned in the final output.

3.2 VISUAL UNDERSTANDING

Initial Table: | department | salary | | :--- | :--- | | IT | 50000 | | IT | 60000 | | HR | 40000 |

After GROUP BY: | department | AVG(salary) | | :--- | :--- | | IT | 55000 | | HR | 40000 |

3.3 IMPORTANT RULE

Every column in the **SELECT** clause must be:

- Either in the **GROUP BY** clause
- Or inside an **aggregate function** (like SUM, AVG, etc.)

4. HAVING CLAUSE

Used to filter grouped data.

4.1 WHY NOT WHERE?

- **WHERE** filters rows **BEFORE** grouping happens.
- **HAVING** filters groups **AFTER** aggregation is calculated.

4.2 BASIC HAVING

SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 50000;

4.3 WHERE vs HAVING (Combined)

```
SELECT department, AVG(salary) FROM employees WHERE salary > 30000 GROUP BY department HAVING  
AVG(salary) > 50000;
```

Execution Order:

1. **WHERE** filters individual rows.
 2. **GROUP BY** groups the remaining rows.
 3. **HAVING** filters the resulting groups.
-

5. CASE STATEMENT (CONDITIONAL LOGIC)

Used to apply "IF-THEN-ELSE" logic within a query.

5.1 BASIC CASE

```
SELECT name, CASE WHEN salary > 60000 THEN 'High' WHEN salary BETWEEN 30000 AND 60000 THEN 'Medium'  
ELSE 'Low' END AS salary_category FROM employees;
```

Explanation:

- **CASE** → Starts logic.
- **WHEN** → Condition.

- **THEN** → Result if true.
- **ELSE** → Fallback result.
- **END** → Closes the logic.
- **AS** → Names the resulting column.

5.2 CASE WITH GROUP BY

```
SELECT department, CASE WHEN AVG(salary) > 60000 THEN 'High Paying' ELSE 'Normal' END AS dept_category  
FROM employees GROUP BY department;
```

6. COMPLETE ANALYTICAL QUERY FLOW

This is how real-world SQL looks:

SQL

```
SELECT department,  
       COUNT(*) AS total_employees,  
       AVG(salary) AS avg_salary
```

```
FROM employees  
  
WHERE salary IS NOT NULL  
  
GROUP BY department  
  
HAVING AVG(salary) > 40000  
  
ORDER BY avg_salary DESC;
```

Line-by-line Explanation:

- **SELECT department** → The grouping column.
 - **COUNT(*)** → Number of employees per group.
 - **AVG(salary)** → Average salary per group.
 - **WHERE** → Removes NULL values before processing.
 - **GROUP BY** → Organizes data by department.
 - **HAVING** → Keeps only high-paying departments.
 - **ORDER BY** → Sorts the final analyzed groups.
-

7. COMMON ERRORS (VERY IMPORTANT)

1. Using aggregate functions in WHERE:

- *Wrong:* `WHERE AVG(salary) > 50000`
- *Correct:* `HAVING AVG(salary) > 50000`

2. Missing GROUP BY column: Trying to select a column that isn't grouped or aggregated.

3. Mixing aggregated and non-aggregated columns incorrectly.

4. Forgetting execution order: (FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY).

8. MINI PROJECT (FINAL PRACTICE)

Tasks:

1. Count employees per department.
 2. Find average salary per department.
 3. Show only departments with avg salary > 50k.
 4. Categorize employees: High (>60k), Medium (30k–60k), Low (<30k).
-

9. THINKING FRAMEWORK (CRITICAL)

Students should now follow this structure:

1. **What data do I need?** → **SELECT**
2. **From where?** → **FROM**
3. **Any filtering?** → **WHERE**
4. **Need grouping?** → **GROUP BY**
5. **Any condition on groups?** → **HAVING**
6. **Any sorting?** → **ORDER BY**

10. FINAL OUTCOME

After completing all 4 sessions, students can:

- Design databases and tables.
- Insert, update, and delete data.
- Write complex queries with conditions.
- Perform advanced filtering and handle NULLs properly.
- Combine datasets and perform analytical aggregations.
- Use conditional logic in queries.